

Pipeline Microsoft Fabric : copier, contrôler, décider, prévenir

Copier un fichier depuis le web, le contrôler, le charger dans une table Delta, recevoir un e-mail avec le bilan. Quatre activités, un notebook de trente lignes, et un pipeline relançable à volonté sans jamais créer un doublon.

1 · COPY

Copier

le fichier du jour
depuis GitHub

2 · NOTEBOOK

Contrôler

valides d'un côté,
rejets de l'autre

3 · CONDITION

Décider

des rejets ou non ?

4 · OUTLOOK

Prévenir

l'e-mail qui convient

4

activités

2

paramètres

30

lignes de notebook

0

doublon à la relance

AUTEUR

Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS

Un espace de travail Fabric sur une capacité, une boîte Outlook Microsoft 365

FICHIERS DU PROJET

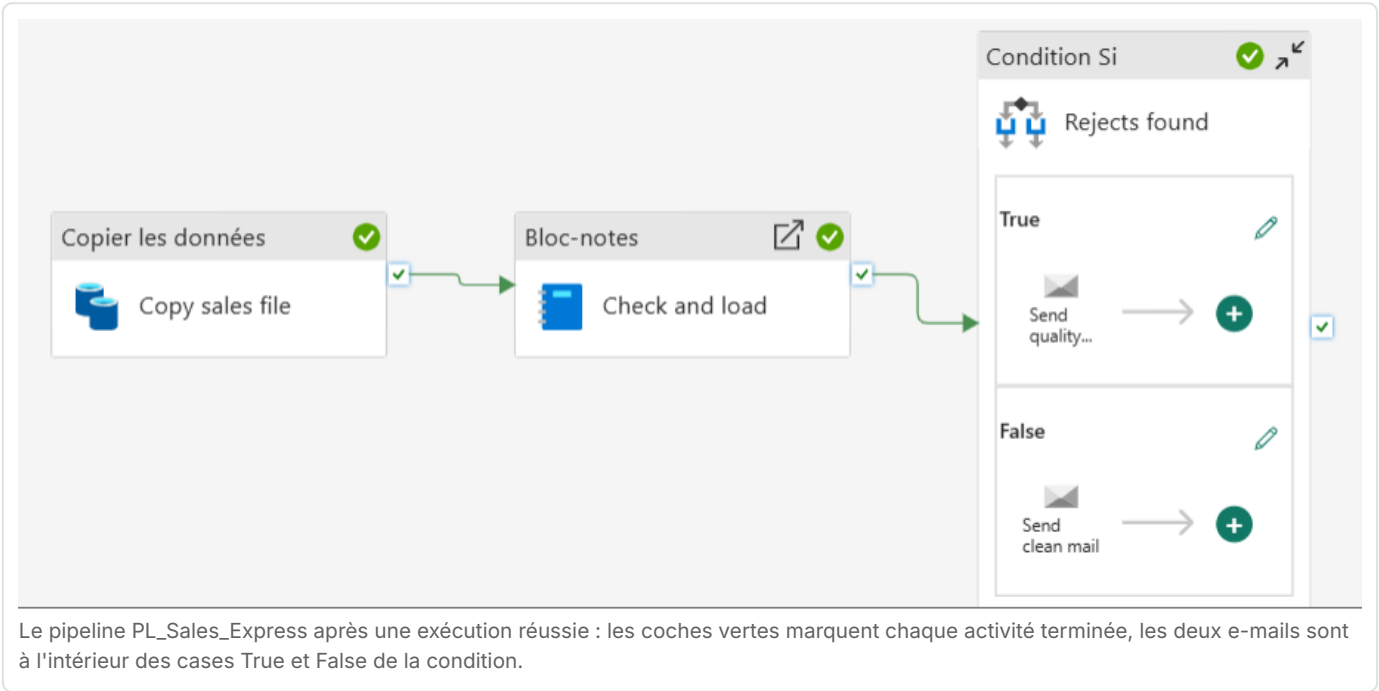
github.com/abdoulhamiddiallo/fabric-pipeline-express

ÉDITION

Octobre 2026

Ce que vous construisez

Chaque soir, une enseigne e-commerce exporte un fichier `sales_<date>.csv`, une ligne par ligne de commande. Comme tout export réel, il contient parfois des lignes fausses. Votre pipeline va le chercher, garde une copie brute, charge les lignes valides, isole les lignes fausses sans les perdre, et vous prévient.



Les objets à créer

LH_Sales (lakehouse), NB_Sales_Quality (notebook, fourni), PL_Sales_Express (pipeline), http_github_sales (connexion).

Ce qu'il vous faut

Un espace de travail Fabric sur une capacité, une boîte Outlook Microsoft 365, et le dossier du projet (lien en couverture).

Cinq idées qui font la différence

IDÉE	OÙ ELLE EST DANS VOTRE PIPELINE	SANS ELLE
Une date en paramètre	<code>load_date</code> choisit le fichier source, le dossier de destination et la partition de la table. Un pipeline pour tous les jours.	Un pipeline par jour, ou une date à modifier à la main.
Le brut est conservé	Le fichier est posé tel quel dans un dossier daté avant tout traitement.	Un bug en aval et la source est perdue.
La qualité isole	Les lignes fausses vont dans <code>sales_rejects</code> avec leur raison. Le pipeline continue.	Le pipeline plante, ou les erreurs polluent le rapport.
Idempotence	L'écriture remplace exactement la date chargée. Relancer ne duplique jamais.	Chaque relance double les chiffres.
Le notebook rend compte	Il renvoie un bilan JSON, le pipeline le lit et décide quel e-mail envoyer.	Le pipeline envoie toujours le même mail, sans savoir ce qui s'est passé.

LE POURQUOI Un pipeline se juge sur ce qu'il fait quand un fichier est faux, quand on doit le relancer, et quand personne ne regarde. Ces cinq idées couvrent exactement ces trois situations.

Les fichiers, le lakehouse, le notebook

OBJECTIF

Tout ce dont le pipeline a besoin, avant de le dessiner

RÉSULTAT

Une URL qui répond, un lakehouse vide, un notebook attaché

A. Les fichiers, accessibles par le web

FICHIER	LIGNES	ANOMALIES VOLONTAIRES	CA VALIDE
data/sales_2026-10-01.csv	1 435	19 : 8 clients absents, 7 quantités négatives, 4 prix à zéro	389 076,55 €
data/sales_2026-10-02.csv	1 347	13 : 4 clients absents, 2 quantités négatives, 7 prix à zéro	369 744,55 €

- 1 Sur GitHub, créez un dépôt public `fabric-pipeline-express` et déposez-y le dossier du projet (branche `main`).
- 2 Testez dans le navigateur : cette adresse doit afficher du texte CSV commençant par `order_id,order_line,order_datetime`.

https://raw.githubusercontent.com/<votre-compte>/fabric-pipeline-express/main/data/sales_2026-10-01.csv

RACCOURCI Le dépôt de l'auteur est utilisable tel quel : <https://raw.githubusercontent.com/abdoulhamiddiallo/fabric-pipeline-express/main/data/>

ATTENTION L'adresse d'une page GitHub (`github.com/.../blob/...`) renvoie du HTML. Seule l'adresse `raw.githubusercontent.com` renvoie le fichier.

B. Le lakehouse

- 1 Espace de travail, **Nouvel élément**, **Lakehouse**, nom `LH_Sales`, case des schémas décochée, **Créer**.

Il contient deux dossiers : **Files** (les fichiers bruts, que la copie déposera) et **Tables** (les tables Delta, que le notebook créera).

C. Le notebook

- 1 Espace de travail, **Importer**, **Notebook**, **À partir de cet ordinateur**, fichier `notebooks/NB_Sales_Quality.ipynb`.
- 2 Ouvrez-le. Volet de gauche, section **Lakehouses**, **Ajouter**, choisissez `LH_Sales`. L'épingle à côté de son nom indique qu'il est le lakehouse par défaut.
- 3 La cellule `load_date = "2026-10-01"` doit porter l'étiquette **Paramètres**. Sinon : menu `...` de la cellule, **Activer/désactiver la cellule de paramètres**.

VÉRIFICATION L'URL répond, `LH_Sales` existe, le notebook affiche l'épingle sur `LH_Sales` et l'étiquette Paramètres sur la cellule `load_date`. N'exécutez pas le notebook : le fichier qu'il lit n'existe pas encore.

Le notebook en quatre idées

Trente lignes, quatre idées. Le notebook porte toute la logique métier ; le pipeline n'en contient aucune.

```
# 1. Lire le fichier déposé par le pipeline, avec des types fixés explicitement
source = f"Files/raw/sales/load_date={load_date}/sales.csv"
df = (spark.read.option("header", "true").csv(source)
      .withColumn("order_datetime", F.to_timestamp("order_datetime"))
      .withColumn("quantity", F.col("quantity").cast("int"))
      .withColumn("unit_price", F.col("unit_price").cast("double"))
      .withColumn("discount_pct", F.col("discount_pct").cast("int"))
      .withColumn("line_amount", F.col("line_amount").cast("double"))
      .withColumn("load_date", F.lit(load_date)))
rows_read = df.count()

# 2. Trois règles de qualité ; la première qui échoue donne la raison du rejet
df = df.withColumn("reject_reason",
  F.when(F.col("customer_id").isNull() | (F.trim("customer_id") == ""), "missing_customer")
  .when(F.col("quantity") <= 0, "quantity_not_positive")
  .when(F.col("unit_price") <= 0, "unit_price_not_positive"))
valid = df.filter(F.col("reject_reason").isNull()).drop("reject_reason")
rejected = df.filter(F.col("reject_reason").isNotNull())

# 3. Écrire de façon idempotente : relancer la même date remplace la même date, rien d'autre
(valid.write.format("delta").mode("overwrite")
  .option("replaceWhere", f"load_date = '{load_date}'").saveAsTable("sales"))
(rejected.write.format("delta").mode("overwrite")
  .option("replaceWhere", f"load_date = '{load_date}'").saveAsTable("sales_rejects"))

# 4. Renvoyer le bilan au pipeline, en JSON
report = {"load_date": load_date, "rows_read": rows_read, "rows_loaded": valid.count(),
  "rows_rejected": rejected.count(), "revenue": round(valid.agg(F.sum("line_amount")).first()[0]
  or 0.0, 2)}
notebookutils.notebook.exit(json.dumps(report))
```

1

Types fixés

Spark devine les types d'un CSV. Un jour il lit 749 et choisit un entier, le lendemain 135.45 et choisit un décimal : une table Delta refuse ce changement. Fixer les types, c'est garantir le même schéma chaque jour.

2

Règles en cascade

`when(...).when(...)` teste les règles dans l'ordre et s'arrête à la première qui échoue. Une ligne n'a qu'une raison de rejet, la plus importante.

3

replaceWhere

Ni *append* (qui duplique à chaque relance), ni *overwrite* (qui efface les autres jours) : on remplace uniquement les lignes de la date chargée.

4

notebook.exit

La valeur passée à `exit` devient la sortie de l'activité Notebook dans le pipeline. C'est ce que lira la condition.

LE POURQUOI Le jour où une règle de qualité change, vous modifiez une ligne de notebook et le pipeline ne bouge pas. Cette séparation entre la logique (notebook) et l'orchestration (pipeline) est la base d'un projet maintenable.

Le pipeline et l'activité Copy

OBJECTIF

Créer PL_Sales_Express, ses paramètres et sa première activité

RÉSULTAT

L'aperçu des données affiche les colonnes du fichier

Le pipeline et ses deux paramètres

- 1 **Nouvel élément, Pipeline de données**, nom PL_Sales_Express, **Créer**.
- 2 Cliquez sur une zone vide du canevas, onglet **Paramètres** en bas, **+ Nouveau** deux fois :

NOM	TYPE	VALEUR PAR DÉFAUT	RÔLE
load_date	Chaîne	2026-10-01	La journée à charger, format aaaa-mm-jj
mail_to	Chaîne	votre adresse e-mail	Destinataire des e-mails

1 Copy sales file

COPIER DES DONNÉES

- 1 Ruban **Activités, Copier des données**. Cliquez sur l'activité, onglet **Général**, Nom = Copy sales file.
- 2 Onglet **Source**, champ **Connexion**, déroulez, **Plus**. Dans la fenêtre, cherchez **HTTP**, cliquez sur la tuile, puis :

URL	https://raw.githubusercontent.com/<votre-compte>/fabric-pipeline-express/main/data/ (barre oblique finale incluse)
Nom de la connexion	http_github_sales
Type d'authentification	Anonyme, puis Connecter
- 3 **URL relative** : cliquez dans le champ, **Ajouter du contenu dynamique**, collez, **OK** :


```
sales_@{pipeline().parameters.load_date}.csv
```
- 4 **Format de fichier** : **DelimitedText**. Puis **Paramètres** à droite, cochez **Première ligne comme en-tête**.
- 5 **Aperçu des données** : les colonnes order_id, order_line ... s'affichent. L'adresse et le format sont bons.
- 6 Onglet **Destination** :

Connexion	LH_Sales
Dossier racine	Fichiers
Chemin, première case	contenu dynamique : raw/sales/ load_date=@{pipeline().parameters.load_date}
Chemin, seconde case	sales.csv
Format de fichier	DelimitedText, Paramètres, Première ligne comme en-tête

- 7 Enregistrez (icône disquette).

ATTENTION Ne cherchez pas HTTP dans le gestionnaire de connexions de la roue dentée : il n'y est pas toujours, et le type GitHub qui s'y trouve (champs « Propriétaire », « Nom du dépôt ») est celui de l'intégration Git. La connexion HTTP se crée depuis l'activité Copy, comme ci-dessus.

LE POURQUOI Le dossier load_date=2026-10-01 suit la convention Hive : un jour, un dossier. Relancer le même jour écrase le même fichier au même endroit. La copie est idempotente, exactement comme le notebook.

L'activité Notebook et la condition

OBJECTIF

Lancer le notebook avec la date, puis lire son bilan

RÉSULTAT

Une condition dont l'expression est acceptée sans message rouge

2 Check and load

NOTEBOOK

- 1 Ruban **Activités, Notebook**. Reliez les activités : glissez la **coche verte** du bord droit de `Copy sales file` jusqu'au Notebook. La flèche verte signifie « en cas de succès ».
- 2 Onglet **Général**, Nom = `Check and load`. Ce nom est utilisé dans sept expressions : C majuscule, un espace avant et après « and ».
- 3 Onglet **Paramètres** : Notebook = `NB_Sales_Quality`. Section **Paramètres de base, + Nouveau** : Nom `load_date`, Type Chaîne, Valeur en contenu dynamique :

```
@pipeline().parameters.load_date
```

ATTENTION Par défaut, Fabric nomme l'activité comme le notebook (`NB_Sales_Quality`). Si vous ne la renommez pas, l'expression de la condition échoue avec « Impossible de référencer la sortie de l'activité Check and load ». Le nom de l'activité et le nom du notebook sont deux choses différentes.

3 Rejects found

CONDITION IF

- 1 Ruban **Activités, Condition If**. Reliez `Check and load` à la condition avec la coche verte.
- 2 Onglet **Général**, Nom = `Rejects found`.
- 3 Onglet **Activités**, champ **Expression, Ajouter du contenu dynamique**, collez, **OK** :

```
@greater(json(activity('Check and load').output.result.exitValue).rows_rejected, 0)
```

Lire l'expression de gauche à droite

MORCEAU	SIGNIFICATION
<code>activity('Check and load').output.result.exitValue</code>	Le texte JSON que le notebook a renvoyé avec <code>notebook.exit</code>
<code>json(...)</code>	Transforme ce texte en objet dont on peut lire les champs
<code>.rows_rejected</code>	Le nombre de lignes rejetées
<code>greater(..., 0)</code>	Vrai si ce nombre est supérieur à zéro

RÈGLE D'OR Dans un champ d'expression (paramètre de notebook, condition), l'expression commence par `@` sans accolades. Dans un champ texte (URL relative, chemin, objet d'e-mail), elle s'écrit entre accolades : `@{...}`. C'est la première cause d'erreur de syntaxe.

VÉRIFICATION Aucun message rouge sous l'expression de la condition, et le bouton **Valider** de la barre d'outils ne signale rien. Enregistrez.

Les deux e-mails, dans les branches

OBJECTIF

Un e-mail d'alerte si des rejets, une confirmation sinon

RÉSULTAT

True et False affichent chacune « 1 activité »

La boîte de la condition contient deux cases, **True** et **False**, chacune avec un bouton **+**. Les e-mails se créent à l'intérieur de ces cases, jamais à côté de la condition.

4 Send quality alert

OFFICE 365 OUTLOOK, CASE TRUE

- 1 Cliquez sur le **+** de la case **True**, choisissez **Office 365 Outlook**. Vous entrez dans la branche avec l'activité déjà placée. Onglet **Général**, Nom = Send quality alert.
- 2 Onglet **Paramètres**. À la première utilisation, **Se connecter** avec votre compte Microsoft 365 et acceptez les autorisations. Les champs s'affichent en anglais :

To	@pipeline().parameters.mail_to
Subject	[Fabric] Ventes @{pipeline().parameters.load_date} : @{json(activity('Check and load').output.result.exitValue).rows_rejected} lignes rejetées

- 3 **Body** est un éditeur de texte enrichi : on y tape le texte fixe et on insère chaque valeur par un jeton. Tapez « Lignes rejetées : », cliquez sur l'icône **éclair** à droite de la zone, onglet **Fonctions**, tapez l'expression sans @, **OK** :

```
json(activity('Check and load').output.result.exitValue).rows_rejected
```

Puis tapez « sur », insérez le jeton `...rows_read`, tapez « lignes. Chiffre d'affaires valide : », insérez `...revenue`, tapez « EUR. Détail dans la table sales_rejects. »

- 4 Revenez au pipeline principal : cliquez sur `PL_Sales_Express` dans le fil d'Ariane au-dessus du canevas.

4 Send clean mail

OFFICE 365 OUTLOOK, CASE FALSE

- 1 **+** de la case **False**, **Office 365 Outlook**, Nom = Send clean mail.
- 2 Paramètres :

To	@pipeline().parameters.mail_to
Subject	[Fabric] Ventes @{pipeline().parameters.load_date} chargées sans rejet
Body	jeton <code>...rows_loaded</code> , tapez « lignes chargées, chiffre d'affaires », jeton <code>...revenue</code> , tapez « EUR. »

- 3 Revenez au pipeline principal, enregistrez, **Valider**.

ATTENTION Si vous ajoutez Outlook depuis le ruban alors que vous êtes sur le canevas principal, l'activité se place à côté de la condition et les cases restent vides : les deux mails partent à chaque exécution, et l'expression affiche « n'est pas un ancêtre ». Le bon réflexe est le **+** à l'intérieur de la case.

VÉRIFICATION Sur le canevas principal, True affiche « 1 activité », False aussi, et rien n'est relié à droite de la condition. Le canevas est identique au schéma de la vue d'ensemble.

Exécuter trois fois

OBJECTIF	RÉSULTAT
Constater que le pipeline se comporte comme prévu, relance comprise	Trois e-mails, 2 750 lignes, zéro doublon

RUN	LOAD_DATE	CE QUI SE PASSE	CE QUE VOUS OBSERVEZ
1	2026-10-01	Premier chargement, les tables sont créées.	E-mail « 19 lignes rejetées ». sales : 1 416 lignes.
2	2026-10-02	Deuxième journée ajoutée.	E-mail « 13 lignes rejetées ». sales : 2 750 lignes.
3	2026-10-01	Relance d'une date déjà chargée : la partition du 1er octobre est remplacée.	Même e-mail qu'au run 1. sales : toujours 2 750 lignes.

- 1

Exécuter, vérifiez load_date et mail_to, OK. L'onglet Sortie s'ouvre : une ligne par activité. Deux à quatre minutes au premier run, le temps que Spark démarre.
- 2

À « Opération réussie », survolez Check and load et cliquez sur l'icône de sortie : exitValue contient les quatre compteurs. Send quality alert est présent, Send clean mail absent : la branche False n'a pas tourné.
- 3

Relancez avec 2026-10-02, puis avec 2026-10-01.

ID d'exécution de pipeline0d35d209-95cf-4728-b059-dd3ab718f50e

État du pipeline✔ Opération réussie

Exporter au format CSV

Insights des erreurs

Filtrer

Options de colonne

Q Filtrer par mot clé4 éléments affichés

L'onglet Sortie après le run 1 : état du pipeline « Opération réussie », quatre activités exécutées.

La preuve par SQL

Ouvrez LH_Sales, basculez en haut à droite sur Point de terminaison d'analytique SQL, Nouvelle requête SQL.

```
SELECT load_date, COUNT(*) AS lignes, ROUND(SUM(line_amount), 2) AS chiffre_affaires
FROM sales GROUP BY load_date ORDER BY load_date;

SELECT load_date, reject_reason, COUNT(*) AS n
FROM sales_rejects GROUP BY load_date, reject_reason ORDER BY load_date, n DESC;
```

LOAD_DATE	LIGNES	CHIFFRE_AFFAIRES	LOAD_DATE	REJECT_REASON	N
2026-10-01	1 416	389 076,55	2026-10-01	missing_customer	8
2026-10-02	1 334	369 744,55	2026-10-01	quantity_not_positive	7
			2026-10-01	unit_price_not_positive	4
			2026-10-02	unit_price_not_positive	7
			2026-10-02	missing_customer	4
			2026-10-02	quantity_not_positive	2

VÉRIFICATION Après le run 3, les deux requêtes renvoient exactement ces chiffres et vous avez reçu trois e-mails dont deux identiques. Le run 3 est le plus important : un pipeline qu'on peut relancer sans réfléchir est un pipeline qu'on peut planifier sans stress.

Les pièges, les réflexes, les expressions

SYMPTÔME	CAUSE	CORRECTION
Le gestionnaire de connexions ne propose pas HTTP, ou demande « Propriétaire » et « Nom du dépôt »	HTTP n'y est pas toujours listé ; le type GitHub affiché est celui de l'intégration Git	Créez la connexion depuis l'activité Copy : Source, Connexion, Plus, recherchez HTTP (étape Copy).
« Impossible de référencer la sortie de l'activité Check and load, car ce n'est pas un ancêtre de l'activité actuelle ou elle n'existe pas »	L'activité Notebook a gardé son nom par défaut, ou l'activité qui contient l'expression n'est pas dans la branche	Renommez l'activité en <code>Check and load</code> . Créez les Outlook avec le + des cases True et False.
Le champ Body refuse le collage de l'expression	Body est un éditeur de texte enrichi	Tapez le texte, insérez chaque valeur par l'éclair, onglet Fonctions, expression sans @ (étape Outlook).
Les deux e-mails partent à chaque exécution	Les Outlook sont à côté de la condition, pas dedans	Supprimez-les, recréez-les avec le + à l'intérieur des cases.
Copy : erreur HTTP 404	URL relative erronée, ou pas de fichier pour cette date	Seules les dates 2026-10-01 et 2026-10-02 existent. Testez l'adresse complète dans le navigateur.
Notebook : « Path does not exist »	Lakehouse par défaut absent, ou chemin de destination différent de celui lu	Épinglez <code>LH_Sales</code> dans le notebook. Vérifiez <code>raw/sales/load_date=...</code> et <code>sales.csv</code> .
Erreur de syntaxe près de @ dans un champ texte	@expr au lieu de @{expr}	Accolades dans les champs texte, pas dans les champs d'expression.

Les six réflexes à retenir

1 Paramétrer Une date pilote le fichier, le dossier et la partition.	2 Conserver le brut Le fichier d'origine, tel quel, dans un dossier daté.	3 Isoler, pas bloquer Les rejets dans une table, avec leur raison.
4 Écrire idempotent <code>replaceWhere</code> : relancer remplace, ne duplique jamais.	5 Faire rendre compte Le notebook renvoie un bilan, le pipeline décide.	6 Nommer avec soin Les noms d'activités sont des identifiants dans les expressions.

Les expressions utilisées

EXPRESSION	USAGE
<code>@pipeline().parameters.load_date</code>	Lire le paramètre (champ d'expression)
<code>sales_@{pipeline().parameters.load_date}.csv</code>	L'interpoler dans un texte (champ texte)
<code>@activity('Check and load').output.result.exitValue</code>	Le JSON renvoyé par le notebook
<code>@json(texte).champ</code>	Parser le JSON et lire un champ

Et après. Plusieurs fichiers par jour : une table de configuration et une boucle ForEach qui appelle ce pipeline en enfant. Ne charger que ce qui a changé : un watermark et un MERGE Delta. Un historique : un journal dans un warehouse. Tout cela est réalisé, avec les mêmes conventions, dans le dépôt `fabric-ecommerce-pipelines` du même auteur. Guide librement partageable avec mention de l'auteur.